Research Intake 2026

Day 32

Advanced Object Detection & Tracking

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

The future of AI is bright, and you can be part of it!

1 From Single-Frame Detection to Video

Day 31 §10.3 closed with a direct, explicit forward reference: tracking maintains a detected object's identity consistently across video frames, building directly on Day 31's per-frame detection output as its raw input signal. This day picks that thread up immediately. Object detection, as Day 31 developed it in full, answers “what objects are in this single image, and where?” for one frame in isolation. The moment the input becomes a video — a sequence of frames rather than one static image — a new question appears that detection alone cannot answer: is the car in frame 47 the same car that was in frame 46, or a different car that happens to occupy a similar position? Object detection has no mechanism for answering this at all, because a detector, by construction, treats every frame as a brand-new image with no memory of what came before.

It is worth being precise about what “no memory” means here, since it is easy to underestimate. A detector's weights are fixed at inference time and its forward pass for frame 47 has literally no access to anything about frame 46 — not the previous frame's pixels, not its detections, not any internal state at all — unless something outside the detector itself is explicitly built to carry that information across the frame boundary. Everything this day covers, from Section 4's classical trackers through Section 7's evaluation framework, is precisely that missing piece: a mechanism, external to the detector, for carrying identity and state information from one frame to the next.

This day's organizing idea, stated plainly before the details: detection (Day 31) plus time equals tracking, and tracking plus semantic interpretation equals activity recognition — this day's practical project. Sections 3 through 7 develop tracking in full, from the classical methods that predate deep learning through the detection-based paradigm that dominates modern practice. Section 8 then takes tracked trajectories one step further, into recognizing what an object (typically a person) is doing across time, which is exactly what PP15 asks students to implement and compare.

> **Callout Box:** *Why does naively running Day 31's detector independently on every frame not already solve this problem? Because independence is exactly the flaw. A detector with no memory of prior frames will, on some frames, miss an object it detected a moment ago (a brief occlusion, a slightly different pose, a confidence score that dips just under threshold) and on other frames detect it again a few frames later — producing boxes that flicker in and out of existence rather than a single, continuous, identity-preserving track. Figure 1 shows this concretely: the left panel shows raw per-frame detections with two frames silently missing the object; the right panel shows the same underlying frames once a tracker is added, carrying the object's identity smoothly through the gaps. Downstream, this flicker is not merely cosmetic — any system counting unique objects, measuring dwell time, or plotting a trajectory over a video will simply produce wrong numbers if it is built on flickering per-frame detections alone.*

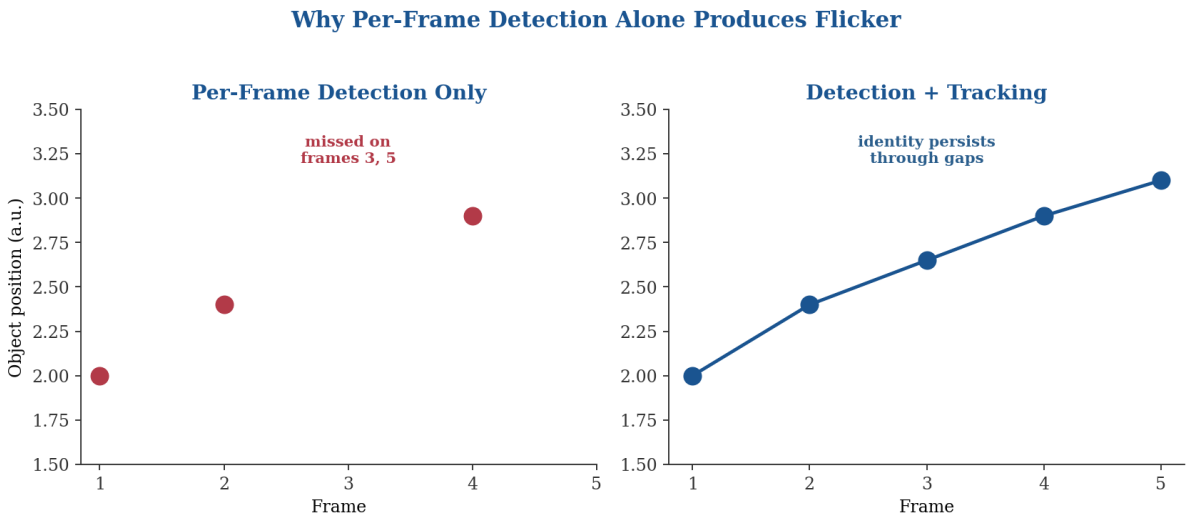

Figure 1. *Independent per-frame detection produces flicker — an object silently disappears and reappears whenever a single frame's detection confidence dips. Tracking (Sections 3–7) carries an object's identity through these gaps rather than re-discovering it from scratch every frame.*

2 Advanced Detection Architectures

It is worth being explicit about why this section exists at all in a day nominally about tracking, rather than assuming Day 31's detector coverage is simply sufficient. Every tracking technique this day develops, from Section 4's classical methods through Section 6's deep trackers, ultimately consumes a stream of per-frame detections as its raw input, and the quality, latency, and structural properties of that upstream detector meaningfully shape which tracking approach is even practical to pair it with — a slow, high-accuracy two-stage detector (Day 31 §4) is a poor match for a real-time tracking application no matter how good the association logic layered on top of it is, since Section 5.1's total pipeline latency is bounded below by the detector's own latency.

Before turning to tracking, it is worth briefly updating Day 31's detection foundations, since tracking-by-detection (Section 5) depends directly on whatever detector produces its per-frame input. Day 31 covered YOLO, SSD, and the R-CNN family in depth; this section is deliberately concise, since Day 31 already built the necessary depth — its job is bridging forward, not re-teaching.

2.1 Anchor-Free Detectors: FCOS and CenterNet

Day 31 §7.2 flagged anchor design as a genuine trade-off: too few anchor shapes hurts accuracy, too many increases cost, and the right anchor set is itself a dataset-specific tuning problem. Anchor-free detectors sidestep this trade-off entirely by removing anchors from the equation. FCOS (Fully Convolutional One-Stage detector) predicts, for every pixel location on a feature map, a direct regression to the four distances from that point to an object's box edges, plus a class label — no anchor boxes to design, cluster, or tune at all. CenterNet takes a related but distinct approach: it predicts an object's center point as a peak on a

heatmap, then regresses the box's width and height directly from that center, treating detection as keypoint estimation rather than as a box-classification problem. Both approaches remove Day 31 §7.1–§7.2's entire anchor-design decision, at the cost of needing careful handling of overlapping objects whose centers or box edges are otherwise ambiguous to assign to a single feature-map location.

It is worth being concrete about when this trade-off is worth taking. Anchor-free detectors tend to simplify the training pipeline meaningfully — there is no anchor-clustering preprocessing step (Day 31 §7.2's dataset-specific anchor shape computation), and no anchor-matching IoU threshold to tune separately from the evaluation threshold (a specific pitfall Day 31 §11 flagged explicitly). This makes anchor-free detectors an increasingly common default choice for new detection pipelines built from scratch, even though, empirically, a well-tuned anchor-based detector and a well-tuned anchor-free detector tend to land within a similar accuracy range on most benchmarks — the appeal is pipeline simplicity as much as any inherent accuracy advantage.

A practical middle ground worth knowing about: several modern detectors combine anchor-free prediction with the multi-scale feature pyramid ideas Day 31 §6.1 introduced for SSD, predicting from several feature-map resolutions simultaneously even without anchors at any one of them. This is not a contradiction of the anchor-free idea — multi-scale prediction and anchor-based prediction are separate design choices that happen to have historically been introduced together in SSD, and can be, and increasingly are, decoupled from one another in newer detector designs.

2.2 DETR and Transformer-Based Detection: A Payoff, Not a Repeat

Day 31 §10.1 introduced DETR's set-prediction reframing of detection — a transformer directly predicting a fixed-size set of objects via a learned bipartite matching, eliminating anchor boxes and Non-Max Suppression together rather than one at a time as the anchor-free detectors above do. That introduction is not repeated here; what is worth adding is a practical framing for this day's purposes: because DETR's output is inherently a clean, non-redundant set of predictions per frame, it removes one specific source of noise (duplicate, NMS-dependent detections) that tracking-by-detection (Section 5) otherwise has to be robust against. This is a genuine, if modest, advantage for tracking pipelines built on top of a transformer-based detector rather than an anchor-based one, and it is one of several reasons detect-and-track research has increasingly moved toward transformer-based joint architectures, previewed further in Section 6.3 and Section 10.1.

This advantage should not be overstated, however. DETR's original formulation, per Day 31 §10.1, was notably slower to converge during training than anchor-based detectors, and that same training cost applies whether DETR is used as a standalone per-frame detector or as the front end of a joint detect-and-track architecture. For PP15's compact, single-session assignment scope specifically, a well-established YOLO or SSD detector (Day 31 §5–§6) remains the more practical choice as the upstream detector feeding Section 5's tracking-by-detection pipeline; DETR-style joint architectures are worth knowing about conceptually

here precisely because they represent where the field is heading, not because they are the right implementation choice for this day's hands-on component.

3 The Object Tracking Problem

It is worth stating a further consequence of Section 1's flicker problem before moving to a formal definition: flicker is not merely an aesthetic annoyance in a displayed video overlay. Any downstream system consuming per-frame detections as if they already carried consistent identity — a counting system, a dwell-time estimator, a trajectory plotter — will silently produce wrong numbers whenever flicker occurs, since it has no way to know that two detections in nearby frames, with no explicit identity, actually refer to the same real-world object. This is precisely why tracking is treated in this compendium as a distinct, necessary layer, not an optional refinement layered on top of "good enough" detection.

With an updated picture of what a modern per-frame detector can supply, this section defines the tracking problem itself precisely, since "tracking" is used loosely in casual conversation but means something specific and testable in this compendium.

3.1 A Formal Definition: Identity Persistence Across Frames

Given a sequence of video frames, object tracking is the task of assigning each detected object a consistent identity (a track ID) that persists correctly across every frame in which that object appears, even through frames where the object is briefly undetected, partially occluded, or changes appearance. This is a strictly harder requirement than Day 31's per-frame detection: a tracker is not merely asked to find objects in each frame independently (Day 31's job), but to correctly link today's detection of "car #3" to yesterday's — or, more precisely, this frame's detection of a given car to the same car's detection one frame earlier — so that a single, continuous trajectory can be recovered for every object in the scene.

A useful way to see exactly how much harder this is: imagine running Day 31's detector on every frame of a ten-second, thirty-frames-per-second clip and getting back three hundred independent sets of boxes, with no labels connecting any box in frame 100 to any box in frame 101. Tracking is the entire process of turning those three hundred independent sets into a much smaller number of continuous trajectories — one per real object in the scene — correctly, even when objects enter, leave, briefly vanish behind one another, or pass close to a visually similar object partway through the clip.

3.2 Single-Object Tracking (SOT) vs. Multi-Object Tracking (MOT)

Two distinct problem settings share the name "tracking" and are worth distinguishing before going further. Single-object tracking (SOT) assumes a single target is specified in the first frame (often by a user-drawn box) and the tracker's job is to follow that one specific object through the rest of the video, with no other objects' identities to manage. Multi-object tracking (MOT), this day's primary focus and the setting Section

7 develops an evaluation methodology for, tracks an unknown, potentially changing number of objects simultaneously — new objects enter the scene, existing ones leave it, and the tracker must manage identity for all of them at once, including deciding when a new detection represents a genuinely new object versus a previously-tracked one reappearing. Section 6.1's Siamese trackers were originally developed for the SOT setting; Sections 5 and 7's tracking-by-detection and MOT-evaluation content assume the harder, more general MOT setting throughout.

It is worth noting why this distinction is more than academic bookkeeping: the two settings favor genuinely different algorithmic designs. SOT can afford a rich, per-target appearance model built and refined from a single, explicitly labeled instance (exactly what Section 6.1's Siamese approach does), since there is only ever one target to model in depth. MOT cannot afford this same depth of per-object modeling scaled up to potentially dozens of simultaneous objects within a real-time budget, which is precisely why the tracking-by-detection paradigm (Section 5) — cheap, detector-driven association rather than an expensive, individually-learned appearance model per object — dominates the MOT setting specifically.

3.3 Core Challenges: Occlusion, Appearance Change, and Camera Motion

Three recurring difficulties distinguish tracking from detection and motivate nearly every technique this day covers. Occlusion — an object temporarily hidden behind another object or the frame edge — breaks the simplest possible tracking strategy (“match to whatever is nearest in the next frame”) outright, since the object simply is not there to match against for some number of frames; Section 4.2's Kalman filter and Section 5.4's SORT baseline exist substantially to handle this gracefully. Appearance change — lighting shifts, pose changes, partial occlusion that alters visible shape — undermines any tracker relying purely on visual similarity between frames, motivating Section 6.2's learned Re-ID embeddings, which are trained specifically to be robust to exactly this kind of variation. Camera motion, finally, is a confound specific to tracking that detection never has to contend with: when the camera itself moves or pans, every object in the frame appears to move even if none of them are actually moving relative to the scene, and a tracker that cannot distinguish object motion from camera motion will produce systematically wrong trajectory estimates — a genuinely separate failure mode from occlusion or appearance change, and one that motion-only trackers (Section 4) are particularly vulnerable to.

These three challenges rarely occur in isolation in real footage, which is part of what makes MOT genuinely difficult in practice rather than merely in theory: a busy street scene combines moving-camera footage (a handheld or vehicle-mounted camera), frequent partial occlusion (pedestrians passing behind one another or behind parked cars), and, for pedestrian tracking specifically, limited appearance variation between different individuals wearing similar clothing — meaning a practical tracker generally needs at least a partial answer to all three challenges simultaneously, not just the one a given technique happens to target most directly.

4 Classical Tracking Methods

Just as Day 31 §4 traced the R-CNN family's incremental refinement, and Day 28 traced classical feature detection's own lineage, this section covers the classical, pre-deep-learning tracking toolkit — not as historical curiosity, but because these methods remain genuinely useful fast baselines even in an era dominated by deep trackers, a point Section 4.4 returns to explicitly.

It is also worth being upfront about a genuine limitation of this section's framing, since it would be misleading to present classical and deep tracking methods as simply competing alternatives at the same level. In practice, the modern tracking landscape is layered rather than binary: Section 4.2's Kalman filter is not a competitor to Section 6's deep Re-ID embeddings at all, but a component that most deep tracking pipelines still use internally for motion prediction, exactly as Section 5.4's spotlight on DeepSORT illustrates. What genuinely competes at the same level are the appearance-modeling choices — correlation filters (Section 4.3) versus learned Siamese or Re-ID embeddings (Section 6) — while motion modeling (Section 4.2) is closer to a shared, largely uncontested foundation running underneath most of this day's higher-level techniques.

4.1 Optical Flow: Lucas-Kanade and Farneback

Optical flow estimates the apparent motion of brightness patterns between two consecutive frames — for every pixel (dense methods) or for a sparse set of tracked feature points (sparse methods), it produces a motion vector describing how that point's location shifted from one frame to the next, illustrated in Figure 2. The Lucas-Kanade method is the classic sparse approach: it tracks a small set of distinctive points (often detected with a corner detector conceptually related to Day 28's classical feature detectors) by assuming brightness constancy and locally constant motion within a small window around each point, then solving a small least-squares system per point. The Farneback method instead computes dense optical flow — a motion vector for every pixel in the frame, not just a sparse set — by approximating each neighborhood's intensity pattern with a polynomial and estimating displacement from how that polynomial changes between frames. Sparse methods are cheaper and sufficient when only specific points (e.g., an object's centroid) need to be tracked; dense methods are more expensive but necessary when the full motion field matters, as in Section 8.2's two-stream activity recognition approach, which uses dense optical flow as a direct input signal.

Both methods share the same underlying assumption worth naming explicitly, since it is also their shared weak point: brightness constancy — the assumption that a given physical point's brightness stays roughly the same between consecutive frames, so that motion can be inferred purely from where a matching brightness pattern moved to. This assumption breaks down under fast lighting changes, reflective or transparent surfaces, and very large inter-frame motion (where the true match falls outside the small search window either method assumes), which is precisely why optical flow alone is rarely used as a complete

tracking solution on its own, but rather as one input signal feeding a larger pipeline — Section 4.2's Kalman filter for motion prediction, or Section 8.2's temporal stream for activity recognition.

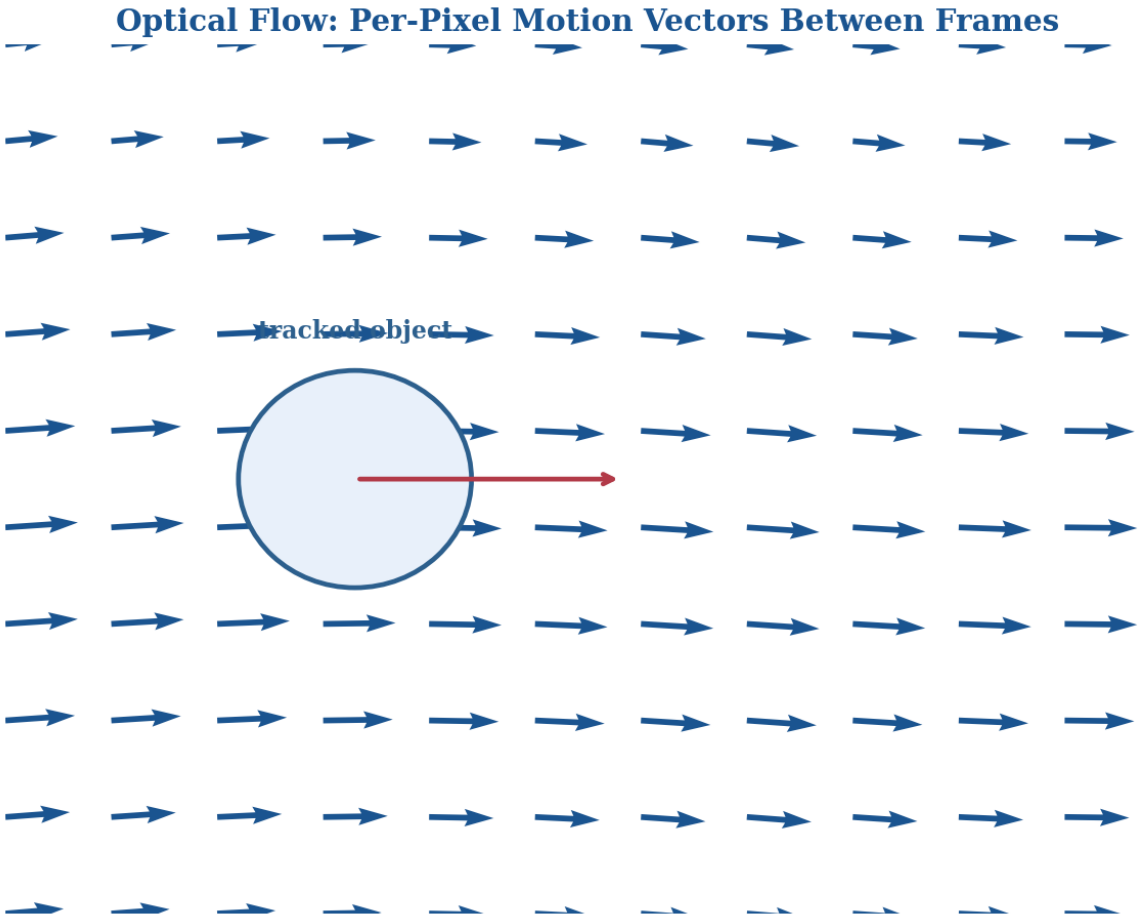

Figure 2. *Optical flow assigns a motion vector to points or pixels between consecutive frames, estimating apparent motion directly from brightness patterns — the geometric basis classical tracking builds on, and later reused as a direct input signal for two-stream activity recognition (Section 8.2).*

4.2 Kalman Filters for State Prediction

A Kalman filter maintains a running estimate of an object's state (typically position and velocity) and updates that estimate in two alternating steps every frame: a prediction step, which projects the current state estimate forward in time using a motion model (e.g., “position next frame equals current position plus current velocity”), and a correction step, which adjusts that prediction using the new frame's actual measurement (e.g., a new detection's box center), weighted by how much the filter trusts the prediction versus the new measurement. This prediction step is precisely what makes Kalman filtering so valuable for Section 3.3's occlusion problem: during frames where an object is occluded and no detection is available at all, the filter can continue predicting the object's likely position from its last known motion, giving the tracker a principled estimate to match against once the object reappears, rather than losing the track entirely the instant a single detection is missed.

The weighting between prediction and measurement is itself worth understanding, since it is the mechanism that makes a Kalman filter behave sensibly rather than either ignoring new detections entirely or discarding its own motion history at every frame. This weighting — the Kalman gain — is computed from the filter's estimate of its own prediction uncertainty relative to the measurement's expected noise: a filter that has recently been receiving consistent, confident detections trusts its motion model less and the new measurement more, while a filter several frames into an occlusion, with growing prediction uncertainty, will trust a plausible new detection quite strongly once one finally reappears. This adaptive weighting is what lets the same Kalman filter framework handle both smoothly-tracked and briefly-occluded objects without any manual switching between modes.

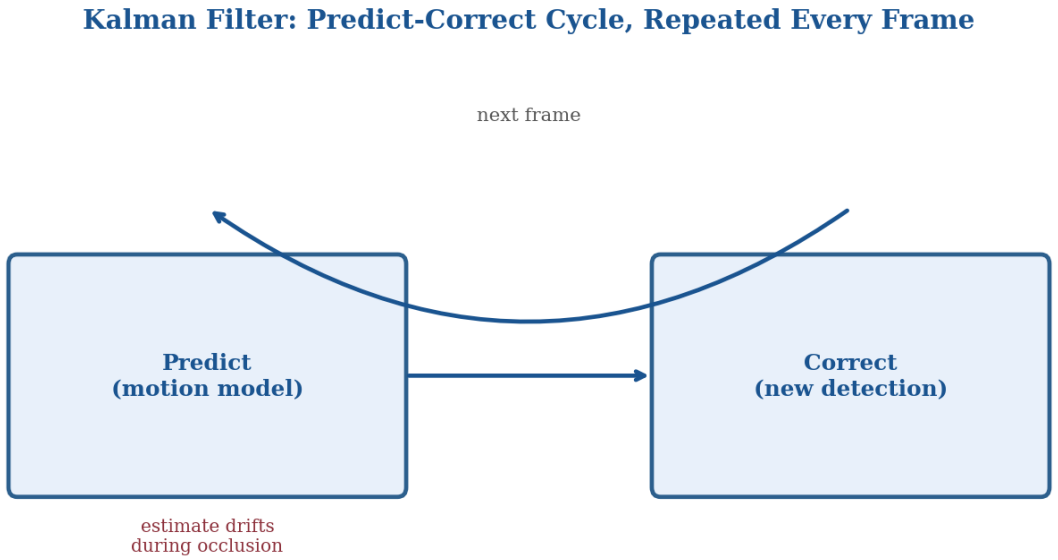

Figure 3. *The Kalman filter's predict-correct cycle repeats every frame: predict forward using the motion model, then correct using the new detection — and continue predicting alone, with growing uncertainty, whenever no detection is available at all.*

4.3 Correlation Filter Trackers: MOSSE and KCF

Correlation filter trackers take a different approach: rather than modeling motion explicitly, they learn a filter — updated online, frame by frame, from the object's own appearance — that produces a sharp correlation peak at the object's location when convolved with a new frame's search region, and a low response everywhere else. MOSSE (Minimum Output Sum of Squared Error) was among the first practical correlation filter trackers, prized for extreme speed (hundreds of frames per second on modest hardware) at the cost of relatively brittle appearance modeling. KCF (Kernelized Correlation Filters) extended this idea with a kernel trick that models a much richer set of implicit training examples without the added computational cost that would normally require, meaningfully improving robustness over MOSSE while retaining most of its speed advantage.

Both trackers exploit a specific mathematical convenience worth understanding at a conceptual level, since it explains their speed advantage directly: correlation, computed in the spatial domain, is an expensive sliding-window operation, but the same computation performed in the frequency domain (via the Fast Fourier Transform) reduces to simple element-wise multiplication, which is dramatically cheaper. Both MOSSE and KCF are built around this frequency-domain trick, which is precisely why they can run at hundreds of frames per second on ordinary CPU hardware where a deep, GPU-dependent tracker (Section 6) cannot — an elegant, purely computational shortcut rather than any inherent simplicity in what they are modeling.

4.4 Why Classical Trackers Remain Relevant as Fast Baselines

It would be a mistake to treat this section as purely historical background ahead of Sections 5–6's deep-learning-based methods. Correlation filter trackers in particular remain the right practical choice whenever compute budget is the binding constraint: MOSSE and KCF run comfortably on CPU at real-time frame rates with no GPU at all, which none of Section 6's deep trackers can claim without dedicated hardware. Kalman filtering, separately, is not really an alternative to deep tracking at all — it is a component used inside nearly every modern tracking-by-detection pipeline (Section 5.1), including deep-embedding-based ones, because state prediction through occlusion is a genuinely separate sub-problem from appearance matching, and Kalman filters remain the standard, well-understood solution to it regardless of how the appearance-matching half of the pipeline is implemented.

A useful rule of thumb when choosing among this day's tracking methods for a new project: reach for a pure correlation filter tracker (Section 4.3) when the target is a single, well-defined object, compute budget is severely constrained, and occasional identity errors are tolerable; reach for SORT (Section 5.4) when multiple objects need tracking under a real-time budget and occlusions are brief; and reach for DeepSORT or a Siamese/Re-ID-based approach (Sections 5.4's spotlight and Section 6) specifically when crowded scenes or longer occlusions make appearance information, not just motion and box geometry, a practical necessity rather than a nice-to-have.

5 Detection-Based Tracking (Tracking-by-Detection)

5.1 The Dominant Modern Paradigm

Tracking-by-detection is, by a wide margin, the dominant paradigm in modern multi-object tracking, and it is the paradigm PP15's video activity recognition pipeline implicitly assumes as its upstream input. Its structure is simple to state: run Day 31's object detector independently on every frame, then solve a separate data association problem — deciding which of this frame's detections correspond to which already-established tracks — frame by frame, using the previous frames' track states as context. This cleanly separates detection quality (Day 31's entire subject) from association quality (this section's subject), which

is precisely why Day 31's detector choice (YOLO, SSD, an anchor-free detector, or DETR) is a genuine, consequential upstream decision for the tracker built on top of it, echoing Day 31 §1.2's backbone-plus-head framing one level up: here, detector-plus-association-strategy is the analogous structural split.

This separation of concerns is also this paradigm's greatest practical strength: because detection and association are decoupled, either half of the pipeline can be improved or swapped out independently. Upgrading from an older detector to a newer, more accurate one (Section 2's anchor-free or transformer-based options, for instance) improves tracking quality without touching the association logic at all; conversely, upgrading from SORT's motion-only association (Section 5.4) to DeepSORT's appearance-aware association (Section 5.4's spotlight) improves tracking quality without retraining or replacing the detector. This modularity is a large part of why tracking-by-detection has remained the dominant paradigm even as detector architectures (Section 2) and association strategies (Sections 5.2–5.4, Section 6) have each evolved substantially and largely independently of one another.

It is worth being concrete about how modest this modularity's real-world cost is, since it is easy to underestimate how much a hand-designed association layer actually adds on top of a detector that is, by Day 31's standards, already doing the computationally heavy lifting. Running the Hungarian algorithm over a cost matrix with even a few dozen tracks and detections takes a small fraction of a millisecond on ordinary hardware, and Section 4.2's Kalman filter update per track is similarly cheap — meaning a tracking-by-detection pipeline's total per-frame cost is overwhelmingly dominated by the detector itself, and the tracking layer riding on top adds comparatively little additional latency, an important practical property for any of Section 9's real-time applications.

5.2 The Data Association Problem and IoU-Based Matching

Given a set of existing tracks (each with a predicted location for the current frame, per Section 4.2's Kalman filter) and a fresh set of detections from the current frame, the data association problem is: which detection, if any, corresponds to which track? The simplest and most common similarity measure for this matching is IoU — Day 31 §2.3's Intersection over Union, computed here between each track's predicted box and each candidate detection's box, rather than between a prediction and ground truth as Day 31 used it. A high IoU between a track's prediction and a detection is strong evidence they represent the same object; a full pairwise IoU computation between all tracks and all detections in a frame produces a cost matrix that Section 5.3's Hungarian algorithm then resolves into a final one-to-one assignment.

It is worth being explicit about what happens at the edges of this matching process, since real footage constantly produces edge cases. A detection with no sufficiently high-IoU match against any existing track is treated as a candidate new object, typically starting a new, tentative track rather than being discarded outright — tentative, because a single stray false-positive detection (Day 31 §3.1's false-positive concept, reappearing here) should not immediately spawn a permanent new identity; most tracking-by-detection implementations require a new track to be confirmed by several consecutive matching detections before it

is treated as fully established. Symmetrically, a track with no matching detection for the current frame is not immediately deleted either — per Section 4.2, it continues coasting on its Kalman-predicted position for some number of frames before being dropped, giving genuinely occluded objects a real chance to reappear and be correctly re-matched rather than losing their identity the instant a single frame's detection is missed.

5.3 The Hungarian Algorithm for Optimal Assignment

Given the cost matrix Section 5.2 describes (one row per track, one column per detection, entries typically set to 1 − IoU so that lower cost means better match), simply assigning each track to its individually highest-IoU detection greedily can produce inconsistent results — two different tracks might both prefer the same detection, and a greedy approach has no way to resolve this optimally. The Hungarian algorithm solves this properly: it finds the globally optimal one-to-one assignment between tracks and detections that minimizes total cost across the whole matrix simultaneously, guaranteeing no better overall assignment exists, rather than settling for whatever a greedy, row-by-row assignment happens to produce. This is the same optimal-assignment algorithm Day 31 §10.1 previewed as a forward reference from DETR's training-time bipartite matching — DETR uses it to match predictions to ground truth during training; tracking-by-detection uses the identical algorithm to match detections to tracks at inference time, a genuinely structurally similar assignment problem appearing in two different places in this module.

It is also standard practice to combine the Hungarian algorithm's optimal assignment with a hard cost threshold: even the best available match for a given track, according to the cost matrix, is rejected as invalid if its cost exceeds a chosen threshold (equivalently, if its IoU falls below a minimum acceptable overlap), preventing the algorithm from forcing an obviously wrong match simply because it happens to be the least-bad option available in a particular frame. This threshold plays a role directly analogous to Day 31 §3.1's true-positive IoU threshold, just applied to track-to-detection association rather than to prediction-versus-ground-truth scoring.

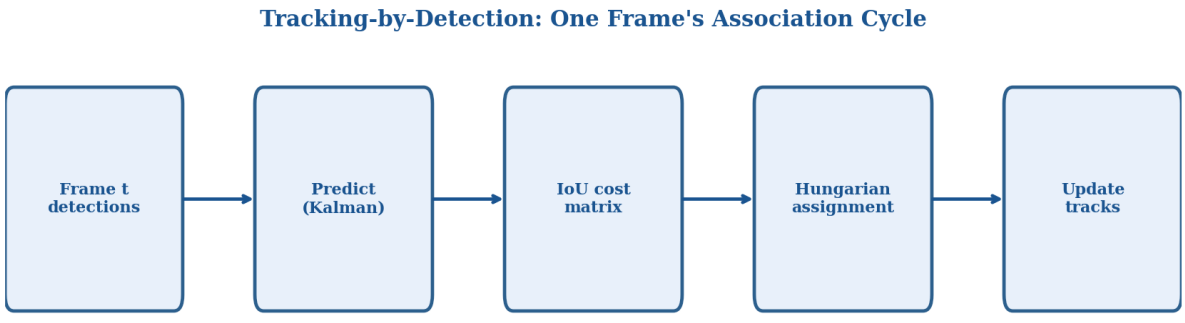

Figure 4. *One frame's tracking-by-detection cycle: detect, predict each existing track's expected position (Section 4.2), build an IoU cost matrix (Section 5.2), resolve it optimally with the Hungarian algorithm (Section 5.3), then update every track's state.*

5.4 SORT: A Minimal Baseline

SORT (Simple Online and Realtime Tracking) combines exactly the three ingredients Sections 4.2, 5.2, and 5.3 developed — a Kalman filter per track for motion prediction, IoU as the sole association similarity measure, and the Hungarian algorithm to resolve the resulting cost matrix — into a complete, minimal, extremely fast tracker with no learned appearance model at all. SORT's central limitation follows directly from what it leaves out: because it matches purely on motion and box overlap, it struggles specifically through long occlusions (where a Kalman-predicted box can drift far enough from a reappearing object's true location that IoU-based matching fails) and in crowded scenes with many similar-looking, closely-spaced objects (where multiple tracks' predicted boxes may all have comparable IoU against several candidate detections). Both limitations point toward the same fix, developed next.

Despite these limitations, SORT's simplicity is genuinely valuable, not merely a historical stepping stone toward DeepSORT. It requires no additional trained model beyond the upstream detector itself, adds negligible per-frame compute cost on top of detection, and is straightforward enough to implement and debug within a single working session — making it a reasonable first tracker to get working end to end for PP15's pipeline before layering on Section 6.2's appearance-embedding refinement, exactly the incremental build-then-improve approach the Common Pitfalls checklist and Tips sections at the end of this day recommend.

> **Research Spotlight:** *DeepSORT extends SORT with exactly the fix Section 5.4 points toward: alongside the Kalman-filter motion cost, it adds a learned appearance embedding (Section 6.2's Re-ID feature, computed once per detection each frame) and combines motion cost with appearance-embedding distance when building the association cost matrix. This single addition substantially improves robustness through occlusion and in crowded scenes, since two objects that briefly overlap in position can still be told apart by what they look like even when their predicted boxes cannot be distinguished by IoU alone — at the cost of the extra compute a learned embedding network requires every frame. In practice, DeepSORT's combined cost is often a weighted sum of the motion-based IoU cost and the embedding-distance cost, letting a pipeline dial in how much to trust each signal depending on scene density.*

6 Deep Learning Trackers

Section 5's DeepSORT spotlight already previewed the core idea this section develops in full: learned, appearance-based similarity, rather than motion and box geometry alone, as the primary signal for maintaining identity across frames.

6.1 Siamese Network Trackers

A Siamese tracker, illustrated in Figure 4, uses two identical, weight-sharing CNN branches: one branch encodes a template crop of the target object (typically taken from the first frame in the SOT setting Section 3.2 described), and the other encodes a larger search region in the current frame. The two resulting feature representations are then compared — commonly via a cross-correlation operation — to produce a similarity

score map over the search region, and the location of the highest similarity score is taken as the object's new position. Because both branches share identical weights, the network learns a single embedding space in which genuinely matching crops of the same object land close together regardless of which branch processed them, which is the core mechanism that lets a Siamese tracker generalize to objects and object categories it never saw labeled examples of during training — it does not need to have been trained specifically to recognize “dogs” or “cars”, only to recognize when two crops depict the same specific object instance.

This generalization property is worth dwelling on, since it is a genuinely different capability from anything Day 31's detectors offer. A Day 31 detector must be trained (or fine-tuned) against a fixed set of object classes before it can detect them at all; a Siamese tracker, once trained on the general “two crops of the same object instance” task across a large and diverse set of objects, can be handed a template crop of an object it has genuinely never seen a labeled example of before and still track it correctly through subsequent frames, because it is solving a fundamentally category-agnostic similarity problem rather than a fixed-category classification problem — a preview, in a much narrower setting, of the same open-set generalization Section 8.2 of Day 33 will develop in far more depth for the Segment Anything Model.

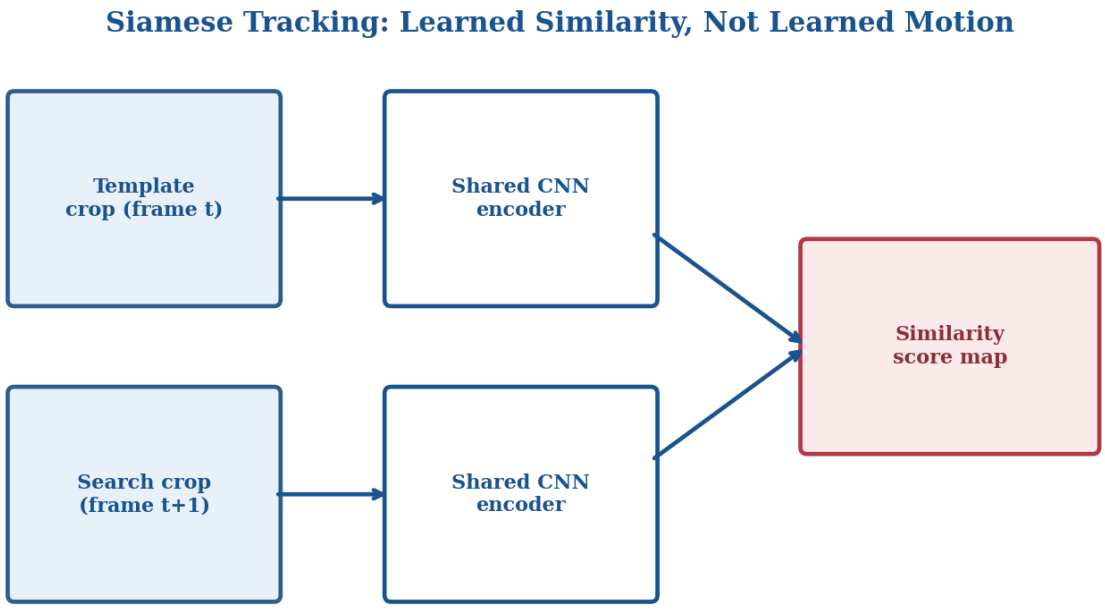

Figure 5. *A Siamese tracker learns similarity, not motion: a shared-weight CNN encodes both a template crop and a search region, and their cross-correlation produces a similarity map whose peak locates the tracked object — the same underlying mechanism DeepSORT's appearance embedding (Section 5.4) draws on.*

It is worth clarifying one further point of terminology before moving on, since “template” and “search region” can otherwise sound like they describe two entirely different kinds of image. Both are simply crops taken from ordinary video frames — the template is usually a tight crop around the target from an earlier frame (often the very first one, in the SOT setting), and the search region is a larger crop from the current

frame, centered on roughly where the target is expected to be based on its previous position, so that the cross-correlation step only has to search a bounded, computationally tractable area rather than the entire frame.

6.2 Re-Identification (Re-ID) Embeddings

Re-identification models are trained with a more explicit, MOT-specific objective than a general Siamese similarity network: given two crops, output embeddings such that crops of the same object instance land close together in embedding space and crops of different objects land far apart, typically trained with a metric-learning loss (e.g., triplet loss) across many identities. This is precisely the appearance-embedding component DeepSORT adds to SORT (Section 5.4), and it is worth being explicit about why a Re-ID embedding succeeds where raw IoU fails: IoU only measures spatial overlap between predicted and observed boxes, which breaks down completely whenever a track's motion prediction is even moderately wrong, while a Re-ID embedding measures what the object actually looks like — information that remains valid and discriminative even when an object's predicted location has drifted substantially, exactly the occlusion and crowded-scene scenarios Section 5.4 identified as SORT's weak points.

Re-ID models are most commonly trained specifically for pedestrian or vehicle re-identification, given how central both categories are to surveillance and traffic-monitoring MOT applications (Section 9), using large datasets of the same individual pedestrian or vehicle photographed from multiple cameras, angles, and times of day, so that the resulting embedding is genuinely robust to the kind of appearance variation Section 3.3 identified as a core tracking challenge, rather than merely memorizing a single canonical viewpoint.

6.3 End-to-End Detect-and-Track Transformers

The most recent line of tracking research collapses Sections 5 and 6's separate detect-then-associate pipeline into a single, jointly trained transformer that performs detection and association together, rather than as two independently optimized stages connected by a hand-designed matching step (the Hungarian algorithm, Section 5.3). These architectures extend Day 31 §10.1's DETR set-prediction framing across time: instead of predicting a fixed set of objects for one frame, they maintain a set of track queries that persist and update across frames, letting the network learn end to end how appearance, motion, and identity should be combined — a direct, concrete instance of Day 30 §7's attention-mechanism preview extended into the temporal dimension, and the clearest illustration yet in this module of the general trend Day 31 §4.3 traced for detection itself: hand-designed pipeline stages progressively give way to a single learned network as enough training data and compute become available.

As with Day 31 §10.1's DETR discussion, it is worth being fair about the trade-off rather than treating end-to-end as an unambiguous improvement: these joint architectures generally require substantially more training data and compute than tracking-by-detection's modular pipeline, precisely because they are learning the association logic from data rather than relying on the well-understood, hand-designed

Hungarian-algorithm-plus-cost-matrix approach Section 5 develops. For most practical deployments today, and certainly for PP15's compact assignment scope, tracking-by-detection remains the more tractable choice; end-to-end detect-and-track transformers are, at present, closer to a research frontier (Section 10.1 returns to this) than a default production recommendation.

7 Multi-Object Tracking (MOT)

It is worth pausing, before the metric definitions themselves, on why a dedicated evaluation framework for tracking could not simply be assembled by running Day 31's detection metrics on every frame and averaging the results. Per-frame averaging treats each frame as an independent measurement, which is precisely the assumption tracking exists to break: two trackers can post identical average per-frame mAP scores while one of them silently reassigns identities every few frames and the other maintains perfectly stable tracks throughout — a difference invisible to any purely per-frame metric, and the entire reason Section 7.1's metrics are defined over the whole sequence rather than frame by frame.

With Sections 4 through 6 having developed classical, detection-based, and deep-learning tracking methods in full, this section synthesizes them into a shared evaluation framework — the same role Day 31 §3 played for detection, now one level up.

7.1 MOT-Specific Evaluation Metrics: MOTA, MOTP, IDF1, and ID Switches

Day 31 §3's mAP measures detection accuracy per frame in isolation and has no way to penalize a tracker for losing an object's identity, so tracking requires its own metric family. MOTA (Multiple Object Tracking Accuracy) combines three per-frame error types — missed detections, false-positive detections, and identity switches — into a single accuracy-style score, directly extending Day 31 §3.1's precision/recall counting logic with a tracking-specific penalty term. MOTP (Multiple Object Tracking Precision) separately measures the average localization precision of correctly matched detections, echoing Day 31 §2.3's IoU-based localization quality but averaged only across true positives rather than folded into the accuracy score. IDF1 measures how well predicted tracks correspond to ground-truth identities across the entire sequence, weighting identity consistency more heavily than MOTA does, which makes it the more informative metric whenever preserving correct identity matters more than raw per-frame detection accuracy — exactly PP15's situation, since Figure 5's illustrative trend shows IDF1 and MOTA can diverge meaningfully between tracker families. ID switches, finally, count a tracking-specific failure mode with no detection-only analogue at all: the number of times a single ground-truth object's assigned track ID changes across the sequence, directly reflecting exactly the flicker-and-reassignment failure Figure 1 illustrated in Section 1.

It is worth being explicit about why both MOTA and IDF1 are typically reported together rather than either alone, since this is a direct, tracking-specific instance of the same multi-metric caution Day 31 §3.4 raised

for AP@0.5 versus AP@[0.5:0.95]. A tracker can achieve a high MOTA largely by having strong per-frame detection quality (few missed or false-positive detections) while still switching identities relatively often in crowded segments, producing a comparatively lower IDF1; conversely, a tracker with a more conservative, identity-preserving association strategy might sacrifice a small amount of raw detection accuracy in exchange for meaningfully fewer identity switches, raising IDF1 at some cost to MOTA. Reporting only one of the two would hide exactly this kind of trade-off from view.

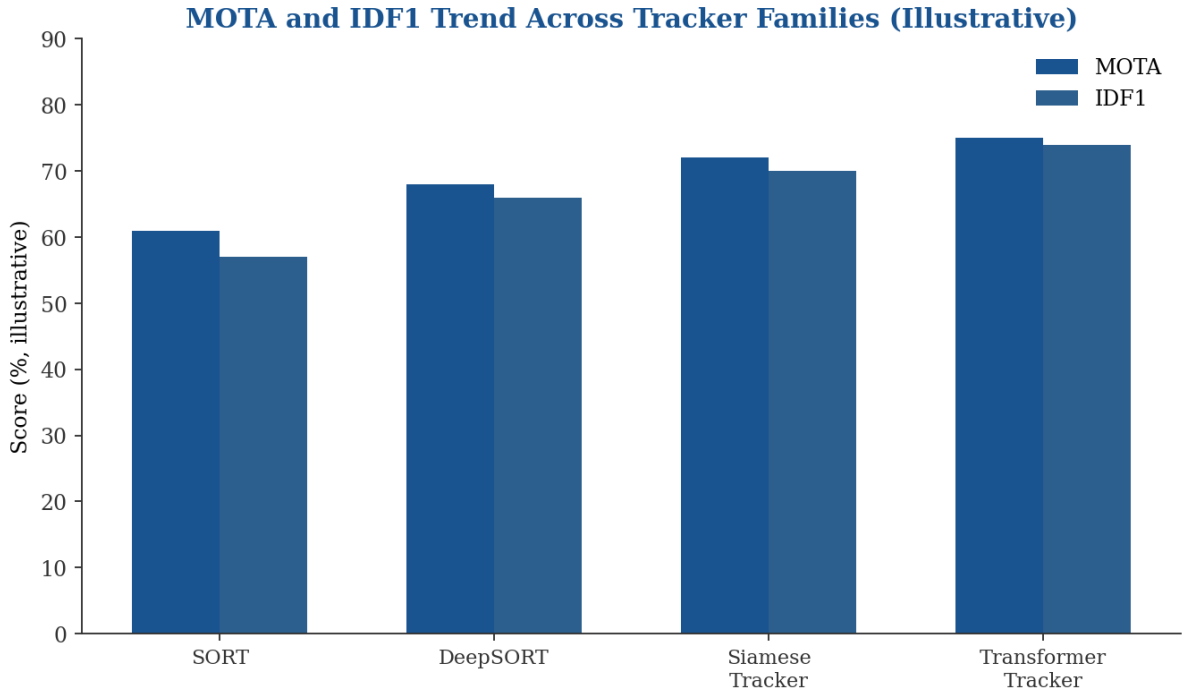

Figure 6. *MOTA and IDF1 both rise from simple motion-only tracking (SORT) toward appearance-aware and transformer-based tracking, but not always in lockstep — a tracker can gain more identity consistency (IDF1) than raw per-frame accuracy (MOTA), or vice versa, which is why both are reported rather than either alone.*

It is worth adding one further metric-design subtlety before moving to benchmarks, since it is a common source of confusion when comparing published numbers. Some MOT evaluation protocols report ID switches as a raw count, while others normalize by the number of ground-truth trajectories or the sequence length, producing numbers that are not directly comparable across papers unless the exact normalization convention is checked — precisely the same “check the convention before comparing” discipline Day 31 §3.4 established for AP@0.5 versus AP@[0.5:0.95], recurring here for MOT metrics specifically.

7.2 Standard MOT Benchmarks

As with Day 31 §3.4's AP@0.5 versus AP@[0.5:0.95] distinction, published MOT numbers are only comparable when computed under the identical benchmark and metric convention. The MOTChallenge benchmark suite (MOT16, MOT17, MOT20, and successors) is the field's standard evaluation setting, providing fixed video sequences with ground-truth annotated tracks specifically so that different tracking

methods can be compared under matched conditions — the same fairness principle Day 31 §8.3 established for comparing YOLO against SSD, now applied to comparing tracking algorithms against each other.

Successive MOTChallenge releases have progressively increased scene density and difficulty specifically to keep pace with tracker capability: MOT16 and MOT17 use moderately crowded pedestrian scenes, while MOT20 specifically targets very high-density crowd scenes where Section 3.3's occlusion and appearance-similarity challenges are pushed to their practical limit. This progression is worth being aware of when reading published tracker comparisons: a strong MOTA or IDF1 score on MOT16 does not necessarily predict similarly strong performance on MOT20's much more crowded scenes, since the two benchmarks are stressing genuinely different aspects of a tracker's design.

7.3 Practical Challenges at Scale

Beyond the metrics themselves, MOT systems face practical challenges that intensify with scene complexity: crowded scenes with many closely-spaced, similar-looking objects (pedestrians in a dense crowd, vehicles in heavy traffic) push exactly the IoU-ambiguity and appearance-similarity limits Sections 5.4 and 6.2 discussed, and long-term occlusion — an object hidden for many consecutive frames, not just one or two — tests whether a tracker's re-identification capability (Section 6.2) is strong enough to correctly re-link a reappearing object to its original track ID rather than silently starting a new one, directly determining the ID-switch count Section 7.1 defined.

A further practical challenge worth naming, distinct from the two above: computational cost at scale. A tracking pipeline that comfortably handles ten simultaneous tracks may struggle to maintain real-time performance with a hundred, since Section 5.2–§5.3's cost-matrix construction and Hungarian-algorithm resolution both scale with the number of tracks and detections involved, and Section 6.2's Re-ID embedding, if used, must be computed for every detection in every frame. Production MOT systems deployed at genuine crowd scale (Section 9's sports and surveillance applications, at their most demanding) typically require deliberate engineering around this scaling cost — batched embedding computation, approximate rather than exact assignment for very large frames — that a compact, single-session implementation like PP15's can reasonably set aside.

8 From Tracking to Activity Recognition

Sections 3 through 7 have fully developed how to track an object's position and identity across frames. This section takes the natural next step, and the one PP15 is built around: once an object — typically a person — has a tracked trajectory across time, what is that object doing? Per-frame classification (Day 29's subject) cannot answer this on its own, because many actions are only distinguishable by their temporal pattern: a single frame of someone with a bent knee could be the middle of a jump, a squat, or simply standing

awkwardly, and only the sequence across frames resolves the ambiguity. Activity recognition therefore needs genuine temporal modeling, not just a stronger per-frame classifier.

8.1 Why Activity Recognition Needs Temporal Context

The core design question every activity recognition approach answers differently is: how should information be aggregated across the frame sequence? A frame-sequence CNN approach folds time directly into the convolution itself — 3D convolutions (Section 8.2's C3D and I3D) slide a kernel across height, width, and time simultaneously, learning spatiotemporal filters that respond to specific motion patterns the same way a 2D convolutional filter (Day 29 §3) responds to specific spatial patterns. A CNN+LSTM/RNN approach instead keeps spatial feature extraction and temporal aggregation as two separate stages: a 2D CNN encodes each frame independently into a feature vector, and a recurrent network processes the resulting sequence of feature vectors, explicitly modeling how they evolve over time. Video transformers, the most recent option, replace the recurrent aggregation step with self-attention across the frame sequence, letting the model learn which frames matter most for a given prediction directly from data rather than through a fixed recurrent update rule.

It is worth being concrete about a specific example that motivates all of this: distinguishing “sitting down” from “standing up” from a single, arbitrarily-chosen frame is close to impossible if that frame happens to catch the person mid-way through either action — the body pose at the midpoint of both actions can look nearly identical. Only the direction of change across consecutive frames (getting closer to the chair versus getting farther from it) actually distinguishes the two actions, which is precisely the kind of signal none of Day 29's single-frame architectures, however accurate on static images, has any way to capture.

It is worth being explicit about the practical implication of this design-question framing before the three-way comparison itself, since it directly shapes how PP15's results should be interpreted. None of the three temporal-aggregation strategies is inherently “correct” in the way, say, Day 31 §2.3's IoU formula is simply correct by definition — each represents a different, reasonable engineering answer to the same underlying question, and which one performs best on a given dataset depends on exactly the kind of actions being classified, how much labeled data is available, and how much compute budget the deployment allows, echoing Day 31 §8.1's “no universal winner” framing for detector families one level up.

8.2 Approaches to Activity Recognition: A Three-Way Comparison

PP15 requires implementing and directly comparing three specific approaches, and this subsection develops exactly the comparison the assignment is built around, illustrated in Figure 6. The 3D CNN approach (C3D, I3D) treats a short video clip as a spatiotemporal volume and learns motion features directly through 3D convolutions, requiring no separate preprocessing step — but at meaningfully higher compute cost than either alternative, since 3D convolutional kernels are substantially more expensive than their 2D counterparts, and requiring correspondingly more training data to learn motion patterns from scratch rather

than relying on a preprocessing shortcut. The two-stream approach runs two separate networks: a spatial stream processing raw RGB frames (much like a standard image classifier, Day 29's subject) and a temporal stream processing Section 4.1's dense optical flow computed between consecutive frames, with the two streams' predictions fused at the end — historically a strong baseline precisely because optical flow hands the temporal stream an explicit, pre-computed motion signal rather than requiring the network to discover motion patterns unsupervised, at the cost of optical flow computation itself becoming a meaningful preprocessing bottleneck (a direct callback to Day 27 §1.1's preprocessing-cost framing). The skeleton/pose-based approach first extracts human pose keypoints per frame using a pretrained pose estimator, then classifies the resulting keypoint trajectory sequence directly — by far the lightest compute cost of the three, since the classifier operates on a small set of joint coordinates rather than raw pixels, and the most robust to background and appearance variation, since clothing, lighting, and scene clutter simply are not part of its input — but it discards any information not captured by joint positions, meaning it cannot distinguish actions that differ only in object interaction (e.g., “drinking from a cup” versus “holding a phone to the ear” might produce near-identical arm-joint trajectories).

It is worth spelling out how the fusion step works in each case, since “fusion” is doing real work in two of the three approaches. In the two-stream approach, fusion is typically a simple weighted average or learned combination of the spatial and temporal streams' final class probabilities — late fusion, in the sense that each stream makes an essentially independent prediction and only the final scores are combined. In the skeleton-based approach, fusion is not really a separate step at all: the keypoint sequence classifier (often itself a small recurrent network or a graph neural network treating joints as graph nodes) already integrates spatial joint layout and temporal evolution jointly within a single model, rather than as two separately-trained streams combined afterward. This structural difference is itself part of what PP15's written analysis (Section D of the assignment brief) should engage with when explaining any observed accuracy differences between the three approaches.

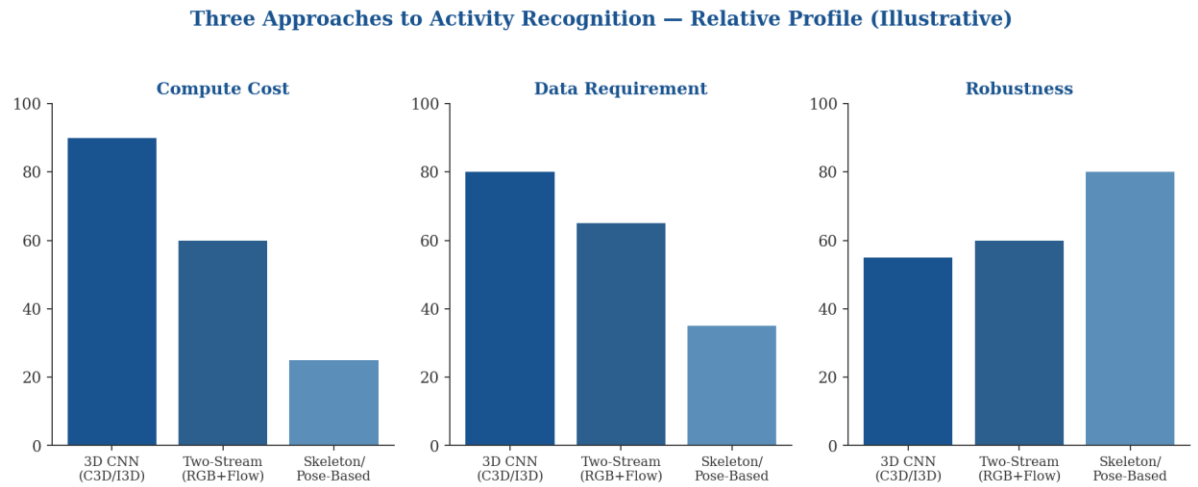

Figure 7. *The three approaches PP15 requires implementing trade compute cost, data requirement, and robustness against each other in different directions — no single approach dominates on all three axes, mirroring Day 31 §8.1's synthesis for detector families one level up.*

9 Real-World Applications

It is worth stating up front what all five applications below have in common structurally, before looking at each individually: every one of them is ultimately a cost-benefit calculation between how much tracking and activity-recognition accuracy a deployment genuinely needs and how much latency, compute, or privacy intrusion it can tolerate to get it — the same underlying calculus Day 31 §9 traced for detection alone, now with a third axis (identity persistence quality, from Section 7's metrics) added to the usual accuracy-versus-latency trade.

The applications below share a common shape: each combines Day 31's detection, this day's tracking, and in several cases Section 8's activity recognition into a single pipeline, with the specific mix and the specific stakes varying by domain.

9.1 Sports Analytics

Sports analytics systems track every player and the ball across broadcast video frames, illustrated in Figure 7, producing per-player trajectories used for tactical analysis (formation shape, pressing intensity, distance covered), automated highlight generation, and broadcast graphics (speed and distance overlays). This is a demanding but comparatively forgiving MOT setting: players are relatively few in number and are generally well-separated on a large field, though Section 7.3's crowded-scene challenges do appear during set pieces and goalmouth scrambles, where many players cluster within a small region and ID switches become noticeably more likely.

Activity recognition (Section 8) layers naturally on top of this tracking foundation in sports analytics specifically: once a player's trajectory is tracked reliably, classifying what that player is doing at each point along it — sprinting, jogging, standing, kicking — turns a raw positional trace into a genuinely tactical dataset, and it is exactly the kind of application where the skeleton/pose-based approach from Section 8.2 tends to be a particularly strong practical fit, since broadcast footage's variable camera angles and crowd backgrounds are exactly the appearance variation pose-based recognition is comparatively robust to.

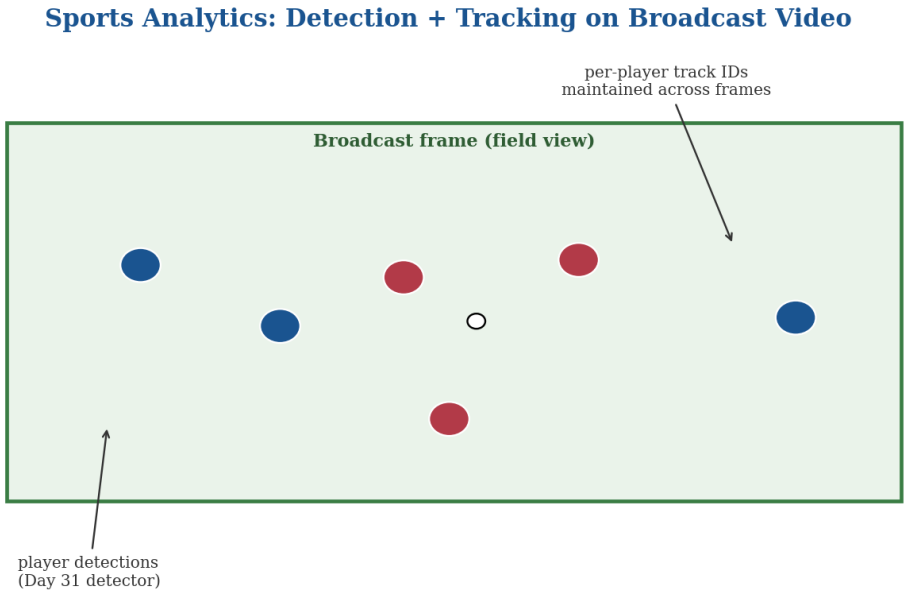

Figure 8. *A sports analytics pipeline: Day 31's detector locates every player and the ball each frame; Sections 5–7's tracking maintains a consistent identity per player across the broadcast, the concrete foundation tactical analytics and highlight generation are built on.*

9.2 Autonomous Driving: Pedestrian and Vehicle Tracking

Day 31 §9.1 established why autonomous vehicle perception cannot tolerate meaningful detection latency. Tracking adds a second, related requirement on top of raw per-frame detection: predicting where a pedestrian or vehicle is likely to move next, which is precisely what Section 4.2's Kalman-filter motion model provides. A vehicle's control system needs not just “where is that pedestrian right now” but “where is that same pedestrian likely to be in the next half-second” to plan a safe trajectory, and answering that question at all requires the identity persistence this day's tracking methods provide — detection alone, re-run independently every frame, has no notion of a pedestrian's velocity or trajectory to extrapolate from.

It is worth naming a specific, recurring difficulty that autonomous-driving tracking faces and sports analytics (Section 9.1) largely does not: sudden, discontinuous appearance from occlusion. A pedestrian stepping out from behind a parked vehicle, or a vehicle emerging from a blind curve, presents a tracker with a genuinely new detection that has no smoothly-predicted Kalman trajectory to match against, since the object was never visible (and therefore never being tracked) in the frames immediately before — precisely the scenario Section 5.2's tentative-track-confirmation logic exists to handle gracefully, converting a fresh, high-stakes detection into a confirmed track as quickly as safety requirements allow.

9.3 Retail Behavior Analysis

Day 31 §9.2 covered retail shelf-inventory monitoring, a detection-only application. Tracking extends retail analytics into behavior: following individual shoppers' movement through a store to understand traffic

patterns, dwell time in specific aisles, and browsing-to-purchase conversion, generally under the same comparatively controlled camera conditions (fixed positions, predictable lighting) Day 31 §9.2 noted made retail detection tractable with lighter-weight models.

9.4 Healthcare: Fall Detection and Patient Monitoring

Section 8's activity recognition finds a direct, safety-relevant application in healthcare monitoring: tracking a patient's position and posture over time to detect a fall (a specific, recognizable activity pattern, per Section 8.1's temporal-context argument) or to monitor mobility and activity levels for elderly or post-surgical patients without requiring continuous staff observation. This application inherits Section 9.5's ethical considerations directly and with particular weight, since continuous monitoring of a person's body and movement in a private setting (a patient's room) raises consent and dignity questions that go beyond the public-space surveillance concerns Day 31 §9.4 raised.

It is worth adding one further, more technical retail-specific consideration before turning to healthcare: because retail behavior analysis often tracks anonymous shoppers (no identity is ever attached to a track ID, only aggregate movement patterns), it sits in a genuinely different position on the privacy spectrum than Section 6.2's Re-ID embeddings, which are explicitly designed to recognize the same specific individual across different cameras and times. A retail deployment using only Section 5's motion-based SORT-style tracking, with no Re-ID component at all, is meaningfully less privacy-invasive than one adding persistent Re-ID — a genuine architectural choice with real privacy consequences, not merely an accuracy-versus-cost trade-off.

9.5 Security and Surveillance: An Ethical Callback

Day 31 §9.4 established that detection-based surveillance, independent of any facial recognition component, already constitutes a meaningful surveillance capability, precisely because it enables the finer-grained behavioral analysis (dwell time, movement pattern, spatial baseline deviation) that a whole-image classifier could not. Tracking and activity recognition intensify this concern directly rather than introducing a new one: a system that can track a specific individual's trajectory across an entire monitored space, and further classify what they are doing at each point along it, produces a substantially richer behavioral profile than static detection alone — the same consent, proportionality, and demographic-fairness auditing considerations Day 26 §9.3 and Day 31 §9.4 raised apply here with added force, and are worth weighing explicitly against this section's legitimate applications (sports analytics, safety-relevant fall detection) rather than treated as a single blanket judgment about tracking technology as a whole.

10 Research Frontiers

10.1 End-to-End Detect-Track-Recognize Pipelines

Section 6.3 introduced transformer architectures that unify detection and association into a single trained network rather than Section 5's separate detect-then-associate pipeline. The active research frontier extends this unification one step further, toward architectures that jointly detect, track, and recognize activity in a single end-to-end trained model, rather than Section 8's current practice of treating activity recognition as a separate stage operating on already-tracked trajectories. Such architectures could, in principle, let gradient signal from an activity label improve the upstream tracking behavior itself — a form of the same "remove the hand-designed intermediate stage" trajectory Day 31 §4.3 traced for region proposals and Section 6.3 traced for detection-association.

It is worth being specific about what "jointly trained" would actually change in practice, since the phrase can otherwise sound like a vague aspiration rather than a concrete research direction. In Section 8's current practice, a tracking error (an ID switch, say) has no way to inform the activity-recognition stage that its input trajectory may be unreliable, and an activity-recognition error has no way to send any corrective signal back to the tracker that produced its input — the two stages are trained and, at inference time, run entirely independently, connected only by one stage's output being fed as the next stage's input. A genuinely end-to-end architecture would instead backpropagate a combined training signal through both stages together, in principle letting the network learn tracking behavior that specifically supports better downstream activity recognition, rather than tracking behavior optimized only against Section 7.1's tracking-specific metrics in isolation.

10.2 Self-Supervised Video Representation Learning

A persistent bottleneck for every deep tracker and activity recognizer this day has covered is the cost of labeled video data — annotating tracked identities and activity labels frame by frame is far more expensive than Day 29's image-level classification labels. Self-supervised video representation learning addresses this by training a network on large volumes of unlabeled video using pretext tasks that require no manual annotation at all (e.g., predicting a shuffled frame sequence's correct order, or learning representations by tracking the same region consistently across frames without any identity labels), then fine-tuning the resulting representation on the comparatively small labeled datasets Section 7.2's benchmarks and PP15's assignment scope actually provide — directly extending Day 29 §7.5's transfer-learning logic from single images into video.

10.3 Forward References: Day 33

This day's tracking content and Day 33's upcoming segmentation content extend Day 31's detection foundation along two genuinely orthogonal axes, illustrated in Figure 8. This day extended detection across time: the same object, a persistent identity, followed frame to frame, while its bounding box representation stayed exactly as Day 31 defined it. Day 33 instead extends detection across space, within a single frame: the same object, but replacing Day 31's bounding box with a precise, pixel-level boundary. These two extensions are independent of each other — a system can track without segmenting, or segment without

tracking — but Day 33 §1 will note that they eventually converge in video object segmentation, which needs both precise per-pixel boundaries and the temporal identity persistence this day developed, tracking a mask rather than a box across frames.

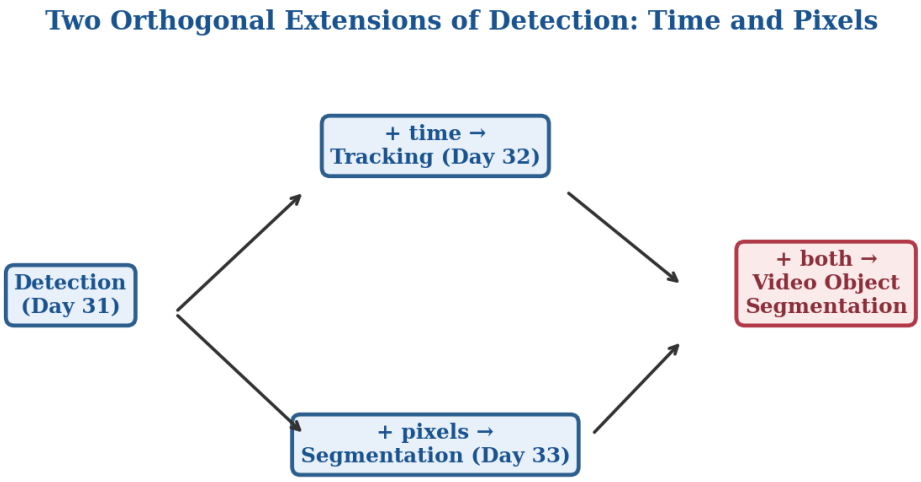

Figure 9. *Detection extends in two independent directions: across time into tracking (this day) and across space into segmentation (Day 33) — and the two combine in video object segmentation, which needs both a precise mask and a persistent identity across frames.*

A further active research direction worth naming alongside Sections 10.1 and 10.2, since it addresses a limitation neither directly solves: multi-camera, multi-view tracking, which maintains a single consistent identity for an object as it moves between multiple cameras' fields of view rather than within one camera's footage alone. This is a genuinely harder version of Section 6.2's Re-ID problem — matching an object's appearance not just across time within one camera, but across different cameras with different viewpoints, lighting, and resolution — and is directly relevant to Section 9.1's sports analytics (tracking a player across multiple broadcast camera angles) and Section 9.5's larger surveillance deployments (a monitored space covered by many cameras with overlapping or adjacent fields of view) alike.

10.4 Closing Perspective

Day 31 developed detection as a single-frame problem in full: given one image, find every object, with what class, and where. This day has taken that single-frame foundation and asked what changes when the input becomes a sequence rather than one image — first identity (Sections 3–7's tracking), then meaning (Section 8's activity recognition). The chain from Day 26's task taxonomy through Day 31's detection and this day's tracking and activity recognition is now complete for the temporal axis; Day 33, immediately following, opens the equally important spatial-precision axis that this day's Section 10.3 previewed. PP15's three-way comparison across activity recognition approaches is where this day's theory becomes a

measured, personal finding, exactly as Day 31 §8.4 framed PP11's YOLO-versus-SSD comparison one day earlier in the sequence.

It is worth carrying one final observation forward from this day specifically, since it recurs throughout the rest of this module: nearly every technique covered here — classical trackers, tracking-by-detection, deep Re-ID, end-to-end transformers — represents a different point along the same speed-versus-robustness spectrum Day 31 §8 established for detector families, now recurring one level up the pipeline. There has been no single “best” tracker in this day any more than there was a single “best” detector in Day 31; the right choice, as always in this compendium, depends on the specific combination of scene density, occlusion severity, and latency budget a given deployment actually faces.

11 Common Pitfalls: A Practical Debugging Checklist

Because this day combines multiple architectural stages and a dense evaluation methodology, a poor result can trace back to any of several places. The following checks catch the most common issues.

- Visualize tracked trajectories overlaid on the actual video before trusting any MOT metric — an ID-switch count or MOTA score alone can hide systematic problems (a track silently jumping between two different real objects) that are immediately obvious once a handful of tracked sequences are watched directly.
- Confirm the detector feeding your tracker is itself producing stable, high-confidence detections before debugging the tracking layer — per Section 5.1, a tracking-by-detection pipeline's association quality is capped by its upstream detector quality, and many apparent tracking bugs are actually Day 31-level detection issues surfacing one layer downstream.
- Check whether IoU-only matching (Section 5.2) is failing specifically in occlusion-heavy or crowded segments before assuming a bug — per Section 5.4, this is SORT's known, expected weak point, not necessarily an implementation error, and is precisely the gap DeepSORT's appearance embedding (Section 5.4's spotlight, Section 6.2) is designed to close.
- Verify optical flow is actually being computed correctly before trusting a two-stream activity recognition result — per Section 8.2, a silently degenerate flow field (e.g., all-zero vectors from a preprocessing bug) will not raise an explicit error but will starve the temporal stream of any real motion signal, producing a model that appears to train while actually relying entirely on the spatial stream.
- Watch for class imbalance across action categories skewing PP15's accuracy metric — echoing Day 31 §11's per-class AP caution, a compact action-recognition dataset/subset often has uneven examples per action class, and per-class accuracy (not only the averaged metric) is worth checking

before concluding one of the three Section 8.2 approaches is weaker than the others on a given dataset.

- Watch for a track-confirmation threshold (Section 5.2) set too low or too high — too low, and single stray false-positive detections spawn short-lived phantom tracks that inflate your false-positive count; too high, and genuinely new objects take noticeably longer to be recognized as confirmed tracks, both of which will visibly distort MOTA and IDF1 in ways that have nothing to do with your association or appearance-embedding logic.

11.1 A Note on Reproducibility

Per Day 28 §11.1, Day 29 §11.1, and Day 31 §11.1's reproducibility guidance, tracking and activity recognition introduce further sources of randomness beyond weight initialization and data augmentation: the Hungarian algorithm's assignment (Section 5.3) is deterministic given its cost matrix, but the cost matrix itself depends on the upstream detector's confidence-threshold and NMS randomness where ties occur, and Section 8's temporal-model training involves the same stochastic optimization sources as any deep network. For PP15's three-way comparison specifically, fixing random seeds across all three approaches' training runs is worth doing explicitly, so that any observed accuracy or robustness difference reflects genuine differences between the approaches rather than run-to-run noise.

11.2 A Note on Comparing Against Published Benchmarks

It can be tempting to compare PP15's own MOT or activity-recognition numbers directly against published MOTChallenge leaderboard results or academic action-recognition benchmark figures. As Day 31 §11.2 cautioned for detection, this comparison is rarely meaningful without significant caveats: published tracking and activity recognition benchmarks are computed on full-scale datasets with extensive training budgets, often using ensembles or heavily tuned hyperparameters, none of which matches PP15's deliberately compact, single-session assignment scope. The meaningful comparison for this assignment is always the three approaches against each other under your own matched conditions (Section 8.2), not any one approach against its own differently-resourced published benchmark result.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Track confirmation — The requirement that a tentative track be matched by several consecutive detections before being treated as fully established, preventing a single stray false positive from spawning a permanent identity (Section 5.2).

Track coasting — Continuing to predict a track's position from its Kalman-filtered motion for a limited number of frames after a detection is missed, before the track is dropped (Section 5.2).

Tracking-by-detection — The dominant modern MOT paradigm: run a detector independently every frame, then solve a separate data-association problem to link detections across frames (Section 5.1).

Data association — The problem of deciding which of a new frame's detections correspond to which already-established tracks (Section 5.2).

Hungarian algorithm — An optimal-assignment algorithm that resolves a cost matrix into the globally best one-to-one matching, used both for track-to-detection assignment (Section 5.3) and DETR's training-time bipartite matching (Day 31 §10.1).

Kalman filter — A predict-then-correct state estimator maintaining an object's position and velocity estimate across frames, particularly valuable through brief occlusions (Section 4.2).

Correlation filter tracker — A tracker learning an online-updated filter whose correlation with a search region peaks at the object's location, exploiting frequency-domain computation for very high speed (Section 4.3).

Re-identification (Re-ID) embedding — A learned appearance representation trained so that crops of the same object instance land close together in embedding space, used to make association robust to occlusion and crowding (Section 6.2).

MOTA / MOTP / IDF1 — The core MOT evaluation metric family: MOTA combines missed detections, false positives, and ID switches into one score; MOTP measures localization precision of correct matches; IDF1 weights identity consistency specifically (Section 7.1).

ID switch — A tracking-specific failure mode: a single ground-truth object's assigned track ID changes incorrectly partway through a sequence (Section 7.1).

Siamese tracker — A tracker using two weight-sharing CNN branches to compare a template crop against a search region via cross-correlation, producing a similarity map (Section 6.1).

3D CNN (C3D / I3D) — An activity-recognition approach applying 3D convolutions across height, width, and time to a video clip directly, at high compute cost (Section 8.2).

Two-stream network — An activity-recognition approach fusing a spatial RGB-frame stream with a temporal optical-flow stream (Section 8.2).

Skeleton/pose-based recognition — An activity-recognition approach classifying a sequence of extracted human pose keypoints rather than raw pixels, lightweight and background-robust but blind to object interactions (Section 8.2).

Chapter Summary: Key Takeaways

- Tracking is detection (Day 31) plus identity persistence across time; naive independent per-frame detection produces flicker, not stable tracks (§1).
- Tracking-by-detection dominates modern practice: run a detector every frame, then solve data association — IoU cost matrix resolved by the Hungarian algorithm — as a separate step (§5.1–§5.3).

- Kalman filters predict through occlusion; learned Re-ID embeddings (DeepSORT) add appearance robustness IoU alone cannot provide (§4.2, §6.2).
- Classical correlation-filter trackers remain the right choice under a hard compute budget — speed comes from a frequency-domain shortcut, not from modeling less (§4.3–§4.4).
- MOT needs its own metrics — MOTA, MOTP, IDF1, and ID switches — because per-frame detection accuracy (mAP) cannot penalize identity loss (§7.1).
- Activity recognition needs genuine temporal modeling; per-frame classification alone cannot disambiguate actions that only differ across a sequence (§8.1).
- The three PP15 approaches — 3D CNN, two-stream, skeleton-based — trade compute cost, data requirement, and robustness against each other, with no universal winner (§8.2).
- Tracking and activity recognition intensify Day 31's surveillance ethics concerns by enabling trajectory- and behavior-level analysis, not just presence detection (§9.5).
- This day extends detection across time; Day 33 extends it across space (pixels) — the two converge in video object segmentation (§10.3).

Assignment / Practical Project Brief

PP15: Video Activity Recognition

★ GRADED ASSIGNMENT

This is a graded assignment. The core requirement: implement all three activity-recognition approaches from §8.2 — 3D CNN, two-stream, and skeleton/pose-based — on the same compact action-recognition dataset, and directly compare them using §8.2's methodology, mirroring the parallel-implementation structure PP12 and PP19 used for their own comparisons.

A Core Requirements

Implement all three approaches from §8.2 (3D CNN / C3D-or-I3D-style, two-stream RGB+optical-flow, and skeleton/pose-based) on the same train/validation split, rather than choosing a single approach — the assignment's purpose is the comparison itself, not any one model in isolation. Each pipeline should include its own appropriate preprocessing (frame extraction and resizing for the 3D CNN approach, optical flow computation for the two-stream approach, pose keypoint extraction for the skeleton-based approach per §8.2) as a documented, reproducible step, since §8.2 established that these preprocessing choices materially affect each approach's compute cost and are themselves part of what should be compared and reported.

B Dataset

Choose a compact action-recognition dataset or subset appropriate for Colab T4 constraints (a handful of action classes, a modest number of clips per class), keeping all three approaches' training tractable within a single working session. Verify the dataset's frame-extraction and (for the pose-based approach) pose-estimator compatibility early, per §8.2's approach-specific input requirements.

C Deliverables

- Three working, trained pipelines (3D CNN, two-stream, skeleton-based) on the same train/val split
- Accuracy and F1 evaluation for all three against the same held-out validation set
- A comparison table/plot: accuracy, training time, inference speed, and data/compute requirements for each approach
- A short written recommendation for when each approach is the right choice, tying the observed results back to §8.2's tradeoff discussion rather than reporting numbers alone

D Grading Rubric

As with PP11 (Day 31), evaluation rigor and comparison quality together carry the largest share of the grade — a submission with three technically working but sloppily-compared models will score notably lower than one with three adequately-trained models compared with real methodological care and a genuine, dataset-grounded recommendation.

E Tips and Common Pitfalls

- Optical flow computation cost can dominate the two-stream approach's total runtime — per §8.2, budget for this explicitly rather than discovering it mid-session.
- Pose estimator failures on occluded or unusual poses will silently degrade the skeleton-based approach's input quality — spot-check extracted keypoints on a handful of frames before trusting downstream classification results.
- Temporal downsampling (using too few frames per clip) can discard the exact motion information §8.1 argued is necessary — verify your clip length actually captures a full action cycle for your chosen classes.
- Class imbalance in action datasets is common — per the Common Pitfalls checklist, check per-class accuracy, not only the averaged metric, before concluding one approach is weaker than another.
- Restart the Colab runtime (or explicitly free GPU memory) between training the three approaches if all are trained in the same notebook session, to avoid out-of-memory errors from residual allocated tensors, echoing PP11's identical GPU-memory tip one day earlier.

F Submission Checklist

- ☐ 3D CNN approach trained and evaluated on the held-out validation split
- ☐ Two-stream approach trained and evaluated, with optical flow computed and spot-checked
- ☐ Skeleton/pose-based approach trained and evaluated, with keypoint extraction spot-checked
- ☐ Comparison table/plot completed with your own measured numbers, not placeholders
- ☐ Written recommendation explicitly connects results to §8.2's compute/data/robustness tradeoffs
- ☐ Code is documented, and a short README explains how to reproduce your results

End of Day 32 — ready for gap review and expansion into full compendium.